

Pixel To Geometry (PTG)

Chad B. Hovey,¹ Anirudh A. Patel, Ryan J. Terpsma

Sandia Injury Biomechanics Laboratory
Sandia National Laboratories²

2020-04-18

¹Corresponding author: chovey@sandia.gov

²The Sandia National Laboratories is a multimission laboratory managed and operated by National Technology and Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International, Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525.

Abstract

Classification of pixel values from human CT scans has been successfully undertaken by this laboratory using the U-Net deep learning algorithm. We have also demonstrated successful conversion of pixels to voxels, as a proxy for finite volume or finite element meshes. We have called these two processes, taken in serial, Pixel To Mesh (PTM).

Within the PTM workflow, we found that the mesh step quickly became intractable when the number of voxels grew past the 10s of millions. Essentially, a map of every pixel (voxel) to a single finite volume or finite element created computational domains of excessively large and equally unnecessary mesh refinement. We briefly considered concepts of element/volume decimation and coalescence, but abandoned these starts because the return on time and resource investments soon appeared significantly less compelling than an alternative approach called isogeometric analysis (IGA).

This document details our investigations using IGA in an alternative workflow that we call Pixel To Geometry (PTG). This alternative seeks to selectively use the categorized pixel values as a scaffold for construction of analytical geometric descriptions in 2D and 3D. Thereafter, we propose using the analytical geometries directly in simulation using IGA, which eliminates the mesh step.

Contents

1	Introduction	3
1.1	Overview	3
1.2	Class Segmentation	4
1.3	Instance Segmentation	7
1.4	Motivation by Analogy	8
2	Non-local Methods	9
2.1	Polynomials	9
2.2	Bézier Curves	11
2.2.1	Bézier Line	11
2.2.2	Bézier Quadratic	11
2.2.3	de Casteljau's Algorithm	14
2.2.4	Bézier Cubic	15
2.2.5	Bernstein Polynomials	15
2.3	Bézier Surfaces	20
3	Local Methods	21
3.1	General Splines	21
3.2	Linear Splines	21
3.3	Quadratic Splines	24
3.4	Cubic Splines	24
3.5	B-Splines	25
	Appendices	29
A	Code Listing	30

Chapter 1

Introduction

In the most general sense, this research strives to create autonomous computational methods to

- **See** axial cross-sections of CT/MR scans (computer vision),
- **Classify** what is seen into basic human anatomy categories (recognition, deep learning), and
- **Build** geometric representations of these anatomies into domains (*e.g.*, voxels, meshes, analytical geometry) suitable for computational analysis (*e.g.*, finite element analysis, finite volume analysis).

In a sense, we wish to combine the skills of a radiologist and an engineer into an automated workflow that can be performed by a computer.

The overarching goal of this effort is to push toward highly personalized medicine, wherein the individual's anatomical geometry, derived from an imaging series, is reconstructed into a high-fidelity, three dimensional (3D) geometric model suitable not only for visualization but also for analysis of a personal response to a mechanically deleterious environment, such as blast, blunt, or ballistic exposure. Common examples of these environments include slip-and-fall accidents and automobile collisions. Less frequent, but particularly relevant to the U.S. warfighter, is exposure to blast overpressure from improvised explosive devices (IEDs) and fragmentation.

In the subsequent sections, we present an overview of the topics covered in this manuscript, followed by details of each.

1.1 Overview

To come.

1.2 Class Segmentation

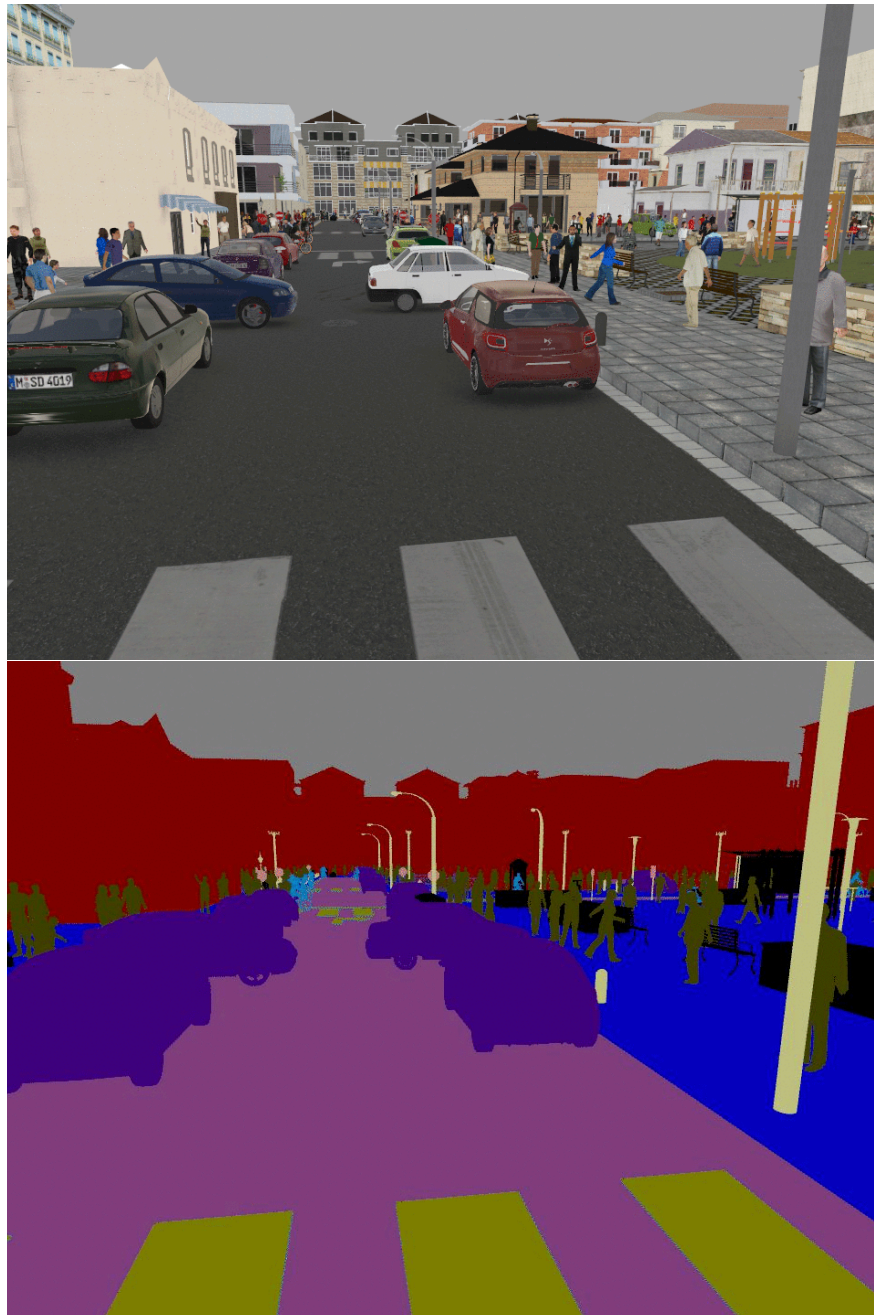


Figure 1.1: Illustration of an image captured into colored pixel values (top) and then color-coded by category, *e.g.*, sky, building, car, road, person, crosswalk (bottom). Reference: [fig/image.pdf](#), [fig/classification.pdf](#), [Heinrich 2016].

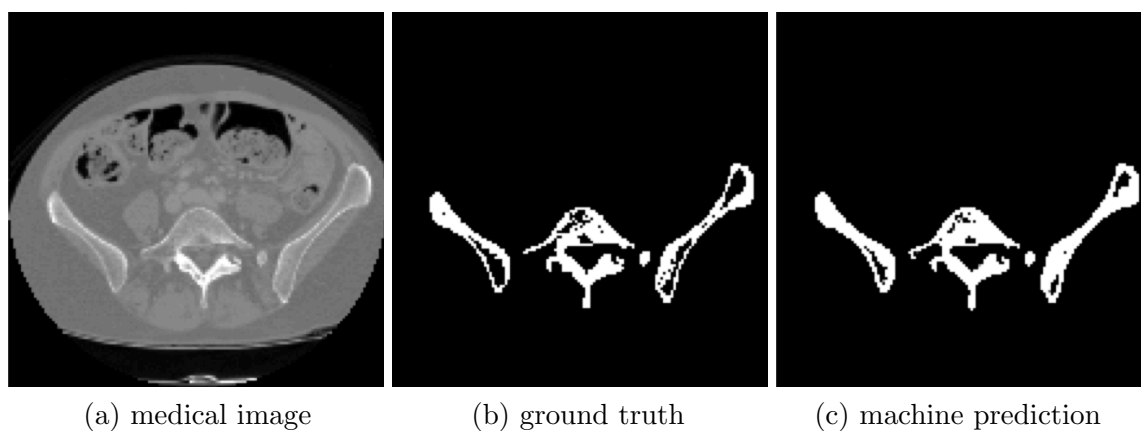


Figure 1.2: Pixel To Mesh (PTM) result of class segmentation, bone versus non-bone, from (a) medical scan; (b) ground truth versus (c) machine prediction. Reference: [fig/ptm_scan.pdf](#), [fig/ptm_ground_truth.pdf](#), [fig/ptm_prediction.pdf](#).

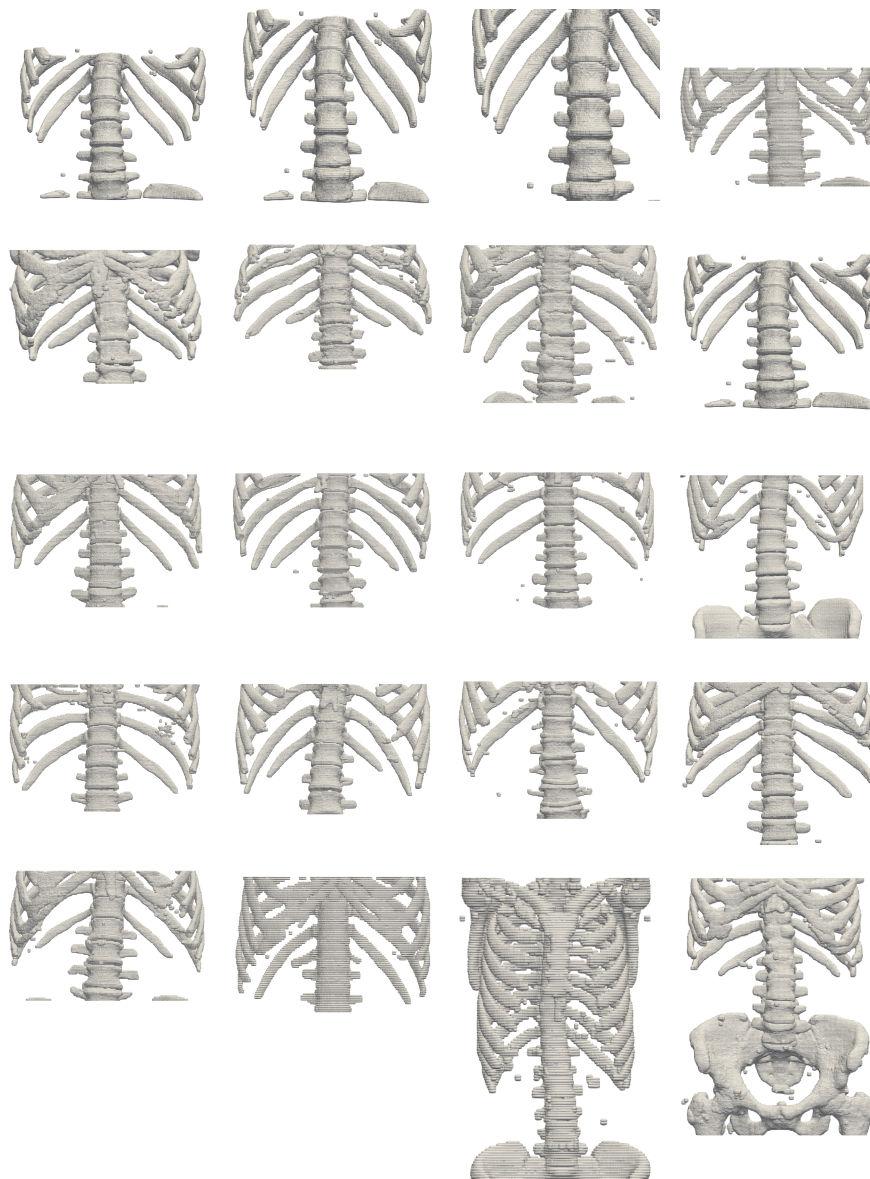


Figure 1.3: Results of the U-NET deep learning algorithm applied [Ronneberger 2015] applied to the IRCAD data set [IRCAD 2019]. Reference: fig/ptm_20_torsos.pdf

1.3 Instance Segmentation



Figure 1.4: A photograph (top) used for (middle) class segmentation, *e.g.*, people versus non-people, and for (bottom) instance segmentation, *e.g.*, a unique person. Reference: [fig/people_photo.pdf](#), [fig/people_class_seg.pdf](#), [fig/people_instance_seg.pdf](#), [Heinrich 2016].

1.4 Motivation by Analogy

The concept of converting pixels to a geometric representation, hereinafter referred to as PTG (Pixel To Geometry), can be introduced by consideration of two similar processes, optical character recognition (OCR) and analytic geometry.

OCR is the process of converting pixel values scanned from a piece of paper with printed or handwritten text into a digital encoding of letters and number, *viz.*

A a B b C c ... Z z 0 1 2 3 4 5 6 7 8 9,

where a collection of pixels is composed to map to letters and numbers.

Figure 1.5 illustrates the difference between vector graphics and rasterized graphics. The word “geometry” appearing with font size 48 point (72 points = 1 inch) composed of vector graphics, and the “e” in yellow, with font size 72 point, appearing from left-to-right at half-inch intervals in rasterized graphics at 10 pixels per inch (ppi), 20 ppi, 40 ppi, and 80 ppi. The image was saved in vectorized format at 600 ppi.

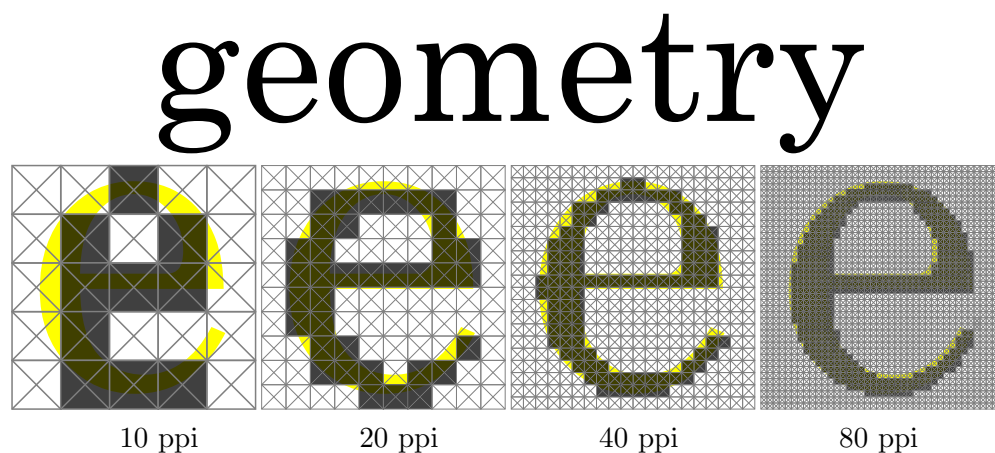


Figure 1.5: Illustration of vector graphics versus rasterized graphics, and the concept of pixel resolution. Reference: [fig/geometry.pdf](#)

Chapter 2

Non-local Methods

In the following sections, we introduce non-local methods to create curves from data points. Thereafter, we introduce local methods. Non-local methods mean that any change in a data point (or a control point) affects the *entire* curve. This is generally undesirable. However, non-local methods are simpler than local methods, so they are introduced first. Table 2.1 presents an overview of the four methods to be covered, categorized by interpolation and locality.

Table 2.1: Categorization of methods by interpolation and locality.

		Locality	
		non-local methods “curves”	local methods “splines”
Interpolation	data points	polynomial	spline
	control points	Bézier, Bernstein	B-spline, NURB

In general, we will refer to non-local methods as **curves** of degree p , denoted $f_p(x) \in \mathbb{R}$ for non-parameterized curves (univariate polynomials, as in Table 2.2) and $C_p(t; \mathbf{x}) \in \mathbb{R}^3$ for parameterized curves (Bézier curves written in the Bernstein basis, as in (2.32)). In contrast, we will refer to local methods as **splines** of degree p , denoted $S_p(t; x) \in \mathbb{R}$ and $S_p(t; \mathbf{x}) \in \mathbb{R}^3$ (used throughout Chapter 3).

2.1 Polynomials

We briefly discuss polynomials for three main reasons: (1) To be explicit regarding the terminology of **power** versus **order** and be clear on the conventions and notations used in this document, (2) to lay the framework for more sophisticated interpolations that will rely on polynomials in some form, and (3) to note shortcomings of polynomials and thus motivate alternative approaches.

First, with regard to *power* versus *order*, [Cottrell 2009] noted the following:

Page 18, Note 3: “There is a terminology conflict between the geometry and analysis communities. Geometers will say a cubic polynomial has degree 3 and order 4. In geometry, order equals degree plus one. Analysts will say a cubic polynomial is order three, and use the term order and degree synonymously. This is the convention we [Cottrell, Hughes, Bazilevs] adhere to.”

Unlike [Cottrell 2009], we adopt the geometry community convention in this document, thus we distinguish between *order* and *power*. We have elected this approach primarily for consistency of this document with historical writings of the geometry community.

Next, there is some basic notation used with polynomials that we need to make clear and concrete. Let $f_p(x)$ be the function of degree p (equivalently, order $p + 1$) that is a sum of variable x raised to some increasing non-negative integer power, starting from zero, and multiplied by a constant coefficient a , such that

$$f_p(x) \triangleq a_0x^0 + a_1x^1 + a_2x^2 + a_3x^3 + \cdots + a_px^p = \sum_{i=0}^p a_ix^i. \quad (2.1)$$

The function $f_p(x)$ is a **polynomial** of **degree** p , the highest non-zero **power**¹ used in the summation. Table 2.2 lists the first five polynomials by order, degree, name, and function. Hereinafter, p will be considered the index of the power of a polynomial. The p -index, a subset of non-negative integers, will start from zero and consecutively increase by one.

Table 2.2: First five orders of a polynomial function.

order	degree	name	function
$\mathcal{O}(1)$	$p = 0$	constant	$f_0(x) = a_0$
$\mathcal{O}(2)$	$p = 1$	linear	$f_1(x) = a_0 + a_1x$
$\mathcal{O}(3)$	$p = 2$	quadratic	$f_2(x) = a_0 + a_1x + a_2x^2$
$\mathcal{O}(4)$	$p = 3$	cubic	$f_3(x) = a_0 + a_1x + a_2x^2 + a_3x^3$
$\mathcal{O}(5)$	$p = 4$	quartic	$f_4(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4$

One note regarding conventions of indices: We use the first item of a series $\mathbf{s}$ to be $\mathbf{s}[0]$, the second item to be $\mathbf{s}[1]$, and so forth. We prefer the zero-based indices because it is consistent not only with our work in Python, which is zero-based, but also with concepts from initial boundary value problems (IBVPs), which denote initial conditions, the first values, as occurring at t_0 .

¹We suggest the mnemonic of “ p ” as the index used for the “power” in a “polynomial.”

Finally, we state as *prima facie* a well-known shortcoming of polynomials is their propensity to overfit the data. This characteristic arises because a polynomial of degree p has $p - 1$ changes of direction, from $-f_p(x)$ to $f_p(x)$, or vice versa. No further discussion on overfitting is given here, though explications of this topic are widely available.

2.2 Bézier Curves

Several of the ideas in this section are due to the excellent references of [Bartels 1995], [Piegl 2012], [Rogers 2000], and [Shiach 2015]. A Bézier curve is a parametric curve defined by control points. A control point $\mathbf{P}_i$ will have two coordinates (x_i, y_i) in 2D and three coordinates (x_i, y_i, z_i) in 3D. The parameter is typically denoted t . The bounds for t are $0 \leq t \leq 1$ unless otherwise indicated.

Here, t does not denote time, as is customary in solid mechanics. Rather, t can be thought of as a “pseudo-time,” wherein the parameterization flows from beginning to end of the t bounds. Also, the bounds of t will later be shown to be arbitrary. For now, however, it is much more convenient to state the bounds as between zero and one.

A Bézier curve of degree p requires $p + 1$ control points. For example, a Bézier curve that is a line (degree $p = 1$) requires two control points. Affine transformations may be used to modify the control points, *e.g.*, to scale, reflect, rotate, or translate (offset) the control points.

2.2.1 Bézier Line

Let $\mathbf{P}(t; \mathbf{x}) \in \mathbb{R}^3$ be **parametric** equation for a line between two points $\mathbf{P}_0 = (x_0, y_0, z_0)$ and $\mathbf{P}_1 = (x_1, y_1, z_1)$, with $\mathbf{x} = (x, y, z)$. Then, let t be the parameter bounded by $0 \leq t \leq 1$. Then the Bézier line between $\mathbf{x}_0$ and $\mathbf{x}_1$ is simply the parameterized linear interpolation $\mathbf{P}_0$ and $\mathbf{P}_1$,

$$\mathbf{P}(t; \mathbf{P}_0, \mathbf{P}_1) \triangleq (1 - t)\mathbf{P}_0 + t\mathbf{P}_1, \quad (2.2)$$

or in explicit coordinate form,

$$\begin{Bmatrix} x(t) \\ y(t) \\ z(t) \end{Bmatrix} = (1 - t) \begin{Bmatrix} x_0 \\ y_0 \\ z_0 \end{Bmatrix} + t \begin{Bmatrix} x_1 \\ y_1 \\ z_1 \end{Bmatrix}. \quad (2.3)$$

2.2.2 Bézier Quadratic

Let $\mathbf{Q}(t; \mathbf{x})$ be the parametric equation for a quadratic between two points $\mathbf{Q}_0$ and $\mathbf{Q}_1$. Let the position of these two points themselves be parameterized by three points $\mathbf{P}_0$, $\mathbf{P}_1$, and $\mathbf{P}_2$, such that

$$\mathbf{Q}_0(t; \mathbf{P}_0, \mathbf{P}_1) = (1 - t)\mathbf{P}_0 + t\mathbf{P}_1, \quad (2.4)$$

$$\mathbf{Q}_1(t; \mathbf{P}_1, \mathbf{P}_2) = (1 - t)\mathbf{P}_1 + t\mathbf{P}_2. \quad (2.5)$$

Thus, the position of Q_0 is on the line between points P_0 and P_1 . The position of Q_1 is on the line between points P_1 and P_2 . At $t = 0$, Q_0 resides at P_0 , Q_1 resides at P_1 . At $t = 1$, Q_0 resides at P_1 , Q_1 resides at P_2 . Finally, let position along the quadratic Bézier curve $Q(t; \mathbf{x})$ be defined as

$$Q(t; Q_0, Q_1) \triangleq (1 - t)Q_0 + tQ_1. \quad (2.6)$$

This curve can be recast in terms of the three points P_0 , P_1 , and P_2 by substituting (2.4) and (2.5),

$$Q(t; P_0, P_1, P_2) = (1 - t)^2 P_0 + 2t(1 - t)P_1 + t^2 P_2. \quad (2.7)$$

Figure 2.1(a) illustrates an example quadratic Bézier curve, with the three control points P_0 , P_1 , and P_2 indicated.^{2,3} We will designate the number of control points to be $(n + 1)$. Thus, the number of control points in the current example (Figure 2.1) is $(n + 1) = 3 \implies n = 2$. Each control point can be identified in sequence as P_i , with $i = 0, 1, \dots, n$. The maximum degree Bézier that can be constructed is two (degree p requires $p + 1$ control points). Thus we see for a series of $(n + 1)$ control points, we can construct a Bézier curve of degree $p = n$.

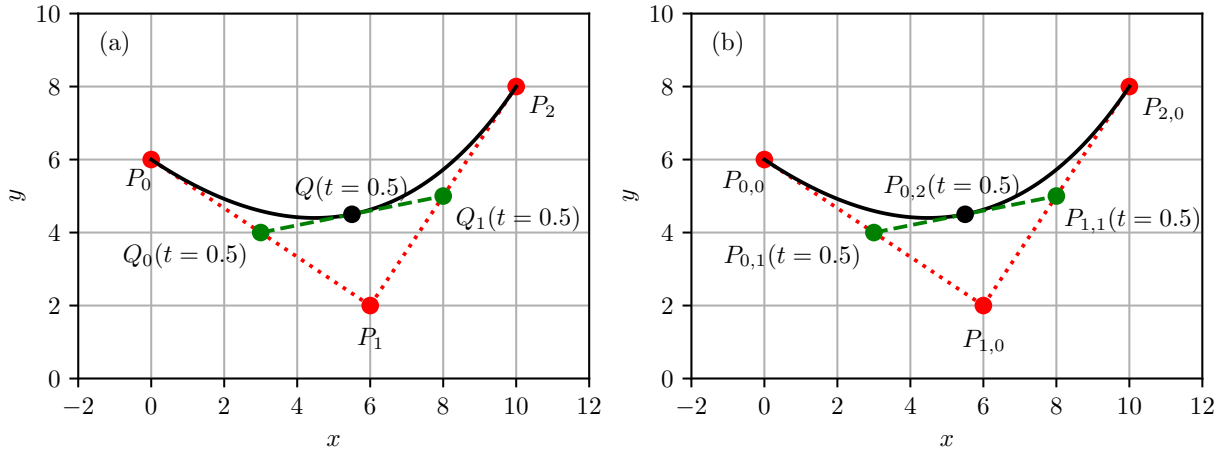


Figure 2.1: A Bézier quadratic curve illustrated at $t = 0.5$ with (a) original notation and (b) the de Casteljau's algorithm notation. Reference: fig/de_casteljau.py.

Example 1.

Figure 2.2 illustrates the same quadratic Bézier curve shown in Figure 2.1, generated at six discrete points in the interval $t \in [0, 1]$. $\square$

²We do not call Q_0 and Q_1 control points, since they are dependent on P_0 , P_1 , and P_2 through parameter t in (2.4) and (2.5).

³Figure 2.1(b) shows the same curve as in Figure 2.1(a), just with a recursive notation, discussed in Section 2.2.3.

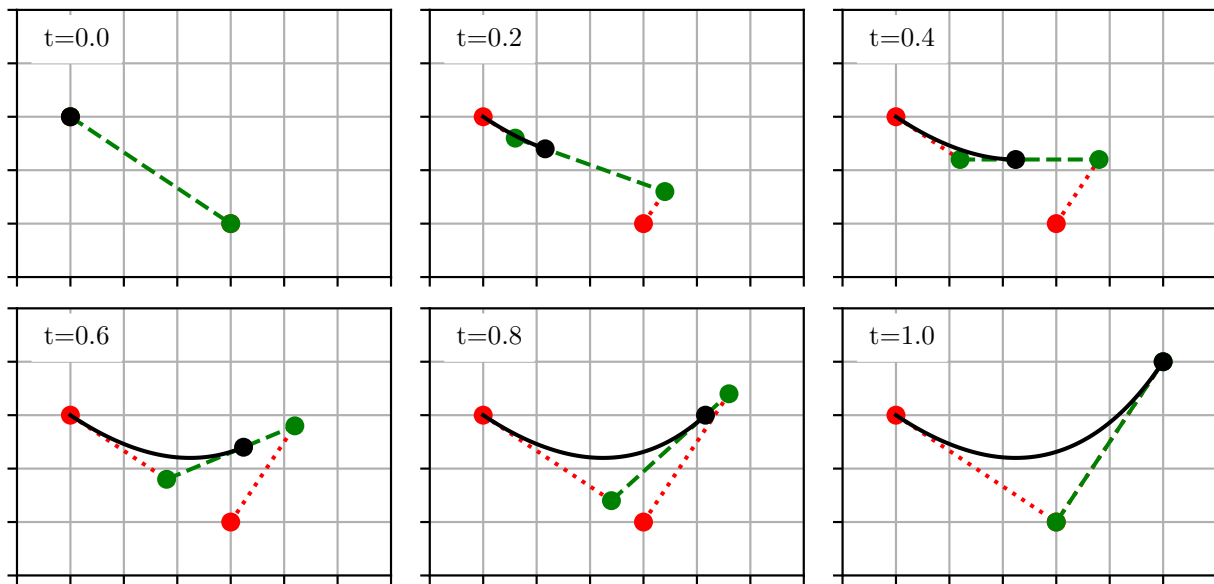


Figure 2.2: The Bézier quadratic curve discussed in Figure 2.1, illustrated in sequence with t starting at 0 and ending at 1. Reference: `fig/de_casteljau.py`.

With these Figures 2.1–2.2 in mind, a few additional observations⁴ can be made:

- The control points sequence P_0, P_1, P_2 is a coarse and discrete *approximation* the continuous quadratic Bézier curve $Q(t)$.
- The end points, P_0 and P_2 , are interpolated, but the interior point P_1 is not interpolated.⁵
- The tangent of the curve $Q(t)$ at $t = 0$ is parallel to the line $P_1 - P_0$. The tangent of the curve $Q(t)$ at $t = 1$ is parallel to the line $P_2 - P_0$.
- The entire curve $Q(t)$ is contained in the triangle formed with vertices P_0, P_1, P_2 .

⁴These items are stated as true but not proven here. Consult the references in the Bibliography section for proofs.

⁵This will generalize to all interior points for higher-degree Bézier curves.

2.2.3 de Casteljau's Algorithm

The foregoing development can be generalized, and is known as **de Casteljau's algorithm**. Let $P_{i,j}$ be control points where

- i denotes the **control point index**, $i = 0, 1, \dots, n$, (total of $n + 1$ control points), and
- j denote the **degree** of the curve, $j = 0, 1, \dots, p$.

Thus $P_{i,0}$ denote the base level control points. In the preceding section,

$$P_0 \mapsto P_{0,0}, \quad j = 0, \text{ thus } P_{i,0} \text{ is a point,} \quad (2.8)$$

$$P_1 \mapsto P_{1,0}, \quad (2.9)$$

$$P_2 \mapsto P_{2,0}, \quad (2.10)$$

$$Q_0 \mapsto P_{0,1}, \quad j = 1, \text{ thus } P_{i,1} \text{ is a line,} \quad (2.11)$$

$$Q_1 \mapsto P_{1,1}, \quad \text{and} \quad (2.12)$$

$$Q(t) \mapsto P_{0,2}, \quad j = 2, \text{ thus } P_{i,2} \text{ is a quadratic.} \quad (2.13)$$

Then, **de Casteljau's algorithm** states

$$P_{i,j} = (1 - t)P_{i,j-1} + tP_{i+1,j-1}. \quad (2.14)$$

Equation (2.7) would then be rewritten as

$$P_{0,2}(t) = (1 - t)P_{0,1} + tP_{1,1}, \quad (2.15)$$

$$= (1 - t)[(1 - t)P_{0,0} + tP_{1,0}] + t[(1 - t)P_{1,0} + tP_{2,0}], \quad (2.16)$$

$$= (1 - t)^2 P_{0,0} + 2t(1 - t)P_{1,0} + t^2 P_{2,0}, \quad (2.17)$$

where only the t parameter is used to describe $P_{0,2}(\cdot)$ for the sake of brevity. Figure 2.1 illustrates for the Bézier quadratic development, (a) the original notation and (b) the de Casteljau's algorithm notation.

For efficiency during computational implementation, it is common to rewrite the quadratic Bézier $P_{0,2}$ in matrix form for **any sequence of three control points**⁶ P_i , P_{i+1} , and P_{i+2} . With

$$P_{0,0} \mapsto P_i, \quad (2.18)$$

$$P_{1,0} \mapsto P_{i+1}, \quad (2.19)$$

$$P_{2,0} \mapsto P_{i+2}, \quad \text{and} \quad (2.20)$$

$$P_{0,2} \mapsto C_2, \quad \text{to denote a curve } C_p \text{ of degree } p = 2, \quad (2.21)$$

⁶This is somewhat of a return map, undoing the notation adopted to explain de Casteljau's algorithm with $i = 0$.

the quadratic Bézier C_2 is written in matrix form as

$$C_2(t) = [P_i \ P_{i+1} \ P_{i+2}] \cdot f(t^2, t, 1), \quad (2.22)$$

$$\begin{Bmatrix} x(t) \\ y(t) \\ z(t) \end{Bmatrix} = \underbrace{\begin{bmatrix} x_i & x_{i+1} & x_{i+2} \\ y_i & y_{i+1} & y_{i+2} \\ z_i & z_{i+1} & z_{i+2} \end{bmatrix}}_{\substack{\text{control points} \\ \text{curve dependent}}} \underbrace{\begin{bmatrix} 1 & -2 & 1 \\ & 2 & 0 \\ \text{sym.} & & 0 \end{bmatrix}}_{\text{curve independent}} \begin{Bmatrix} t^2 \\ t \\ 1 \end{Bmatrix}. \quad (2.23)$$

2.2.4 Bézier Cubic

The cubic Bézier curve $P_{0,3}(t)$, given four control points $P_{0,0}$, $P_{1,0}$, $P_{2,0}$, and $P_{3,0}$, is given by

$$P_{0,3}(t) = (1-t)^3 P_{0,0} + 3t(1-t)^2 P_{1,0} + 3t^2(1-t) P_{2,0} + t^3 P_{3,0}. \quad (2.24)$$

Again for efficiency, it is common to rewrite the cubic Bézier $P_{0,3}$ in matrix form for **any sequence of four control points** P_i , P_{i+1} , P_{i+2} and P_{i+3} . With

$$P_{0,0} \mapsto P_i, \quad (2.25)$$

$$P_{1,0} \mapsto P_{i+1}, \quad (2.26)$$

$$P_{2,0} \mapsto P_{i+2}, \quad (2.27)$$

$$P_{3,0} \mapsto P_{i+3}, \text{ and } \quad (2.28)$$

$$P_{0,3} \mapsto C_3, \text{ to denote a curve } C_p \text{ of degree } p = 3, \quad (2.29)$$

the cubic Bézier C_3 is written in matrix form as

$$C_3(t) = [P_i \ P_{i+1} \ P_{i+2} \ P_{i+3}] \cdot f(t^3, t, 1), \quad (2.30)$$

$$\begin{Bmatrix} x(t) \\ y(t) \\ z(t) \end{Bmatrix} = \underbrace{\begin{bmatrix} x_i & x_{i+1} & x_{i+2} & x_{i+3} \\ y_i & y_{i+1} & y_{i+2} & y_{i+3} \\ z_i & z_{i+1} & z_{i+2} & z_{i+3} \end{bmatrix}}_{\substack{\text{control points} \\ \text{curve dependent}}} \underbrace{\begin{bmatrix} -1 & 3 & -3 & 1 \\ & -6 & 3 & 0 \\ & & 0 & 0 \\ \text{sym.} & & & 0 \end{bmatrix}}_{\text{curve independent}} \begin{Bmatrix} t^3 \\ t^2 \\ t \\ 1 \end{Bmatrix}. \quad (2.31)$$

2.2.5 Bernstein Polynomials

The general form of a degree p Bézier curve $C_p(t)$ defined by $p+1$ control points P_i , $i = 0, 1, \dots, p$, is given by

$$C_p(t) \triangleq \sum_{i=0}^p b_{i,p}(t) P_i, \quad (2.32)$$

where $b_{i,p}(t)$ is a **Bernstein polynomial**, defined as

$$b_{i,p}(t) = \binom{p}{i} t^i (1-t)^{p-i}, \quad \text{and where} \quad \binom{p}{i} = \frac{p!}{i! (p-i)!} \quad (2.33)$$

is the **binomial coefficient**. The binomial coefficients are readily attained from **Pascal's triangle**, written up to $p = 4$ in Table 2.3.

Table 2.3: First five degrees of binomial coefficients from Pascal's triangle.

degree	binomial coefficient				
$p = 0$	1				
$p = 1$	1	1			
$p = 2$	1	2	1		
$p = 3$	1	3	3	1	
$p = 4$	1	4	6	4	1

Using (2.32), the quadratic ($p = 2$), cubic ($p = 3$), and quartic ($p = 4$) Bézier curves can be written, respectively, in explicit form as

$$\mathbf{C}_2(t) = b_{0,2}\mathbf{P}_0 + b_{1,2}\mathbf{P}_1 + b_{2,2}\mathbf{P}_2, \quad (2.34)$$

$$\mathbf{C}_3(t) = b_{0,3}\mathbf{P}_0 + b_{1,3}\mathbf{P}_1 + b_{2,3}\mathbf{P}_2 + b_{3,3}\mathbf{P}_3, \quad (2.35)$$

$$\mathbf{C}_4(t) = b_{0,4}\mathbf{P}_0 + b_{1,4}\mathbf{P}_1 + b_{2,4}\mathbf{P}_2 + b_{3,4}\mathbf{P}_3 + b_{4,4}\mathbf{P}_4. \quad (2.36)$$

Observations⁷ include (see Figure 2.3 for a visual illustration of these properties):

- The Bernstein polynomials always sum to one,

$$\sum_{i=0}^p b_{i,p}(t) = 1.0. \quad (2.37)$$

This concept is called **partition of unity**.

- The first basis is unity $b_{0,p}(0) = 1$ at the start of the interval $t = 0$, when all other bases are zero. Similarly, the last basis is unity $b_{p,p}(1) = 1$ and the end of the interval $t = 1$, when all other bases go to zero.

- The polynomials are **non-negative**,

$$b_{i,p}(t) \geq 0 \text{ for all } \{i, p\} \text{ with } t \in [0, 1]. \quad (2.38)$$

- Each polynomial $b_{i,p}(t)$ has a single maximum in the parameter space $t \in [0, 1]$ at $t = i/p$.
- All polynomials are symmetric in t about $t = 1/2$.

⁷See Bibliography for references containing proofs.

Example 2.

The Bernstein polynomials for a cubic ($p = 3$) Bézier curve are

$$b_{0,3}(t) = \binom{3}{0} t^0 (1-t)^{3-0} = (1-t)^3, \quad (2.39)$$

$$b_{1,3}(t) = \binom{3}{1} t^1 (1-t)^{3-1} = 3t(1-t)^2, \quad (2.40)$$

$$b_{2,3}(t) = \binom{3}{2} t^2 (1-t)^{3-2} = 3t^2(1-t), \quad (2.41)$$

$$b_{3,3}(t) = \binom{3}{3} t^3 (1-t)^{3-3} = t^3, \quad (2.42)$$

which match the coefficients in (2.24). These polynomials are shown in Figure 2.3(c). $\square$

Example 3.

The Bernstein polynomials for a quartic ($p = 4$) Bézier curve are

$$b_{0,4}(t) = \binom{4}{0} t^0 (1-t)^{4-0} = (1-t)^4, \quad (2.43)$$

$$b_{1,4}(t) = \binom{4}{1} t^1 (1-t)^{4-1} = 4t(1-t)^3, \quad (2.44)$$

$$b_{2,4}(t) = \binom{4}{2} t^2 (1-t)^{4-2} = 6t^2(1-t)^2, \quad (2.45)$$

$$b_{3,4}(t) = \binom{4}{3} t^3 (1-t)^{4-3} = 4t^3(1-t), \quad (2.46)$$

$$b_{4,4}(t) = \binom{4}{4} t^4 (1-t)^{4-4} = t^4. \quad (2.47)$$

These polynomials are shown in Figure 2.3(d). Figure 2.4 shows the Bernstein polynomials for $p = 5, 6, 7, 8$. $\square$

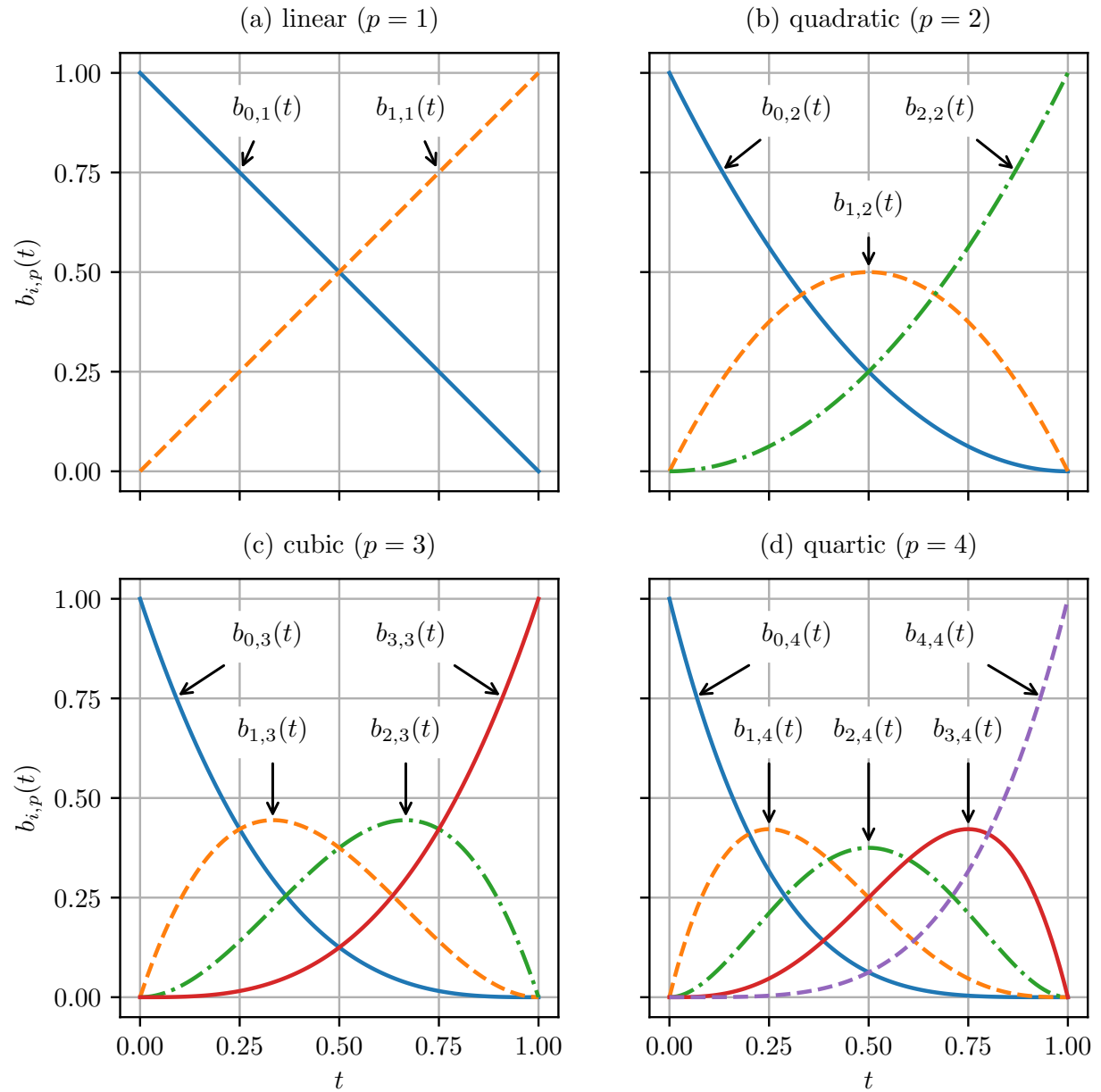


Figure 2.3: Bernstein polynomials for Bézier (a) linear, (b) quadratic, (c) cubic, and (d) quartic curves. Reference: `fig/bernstein.py`, `bernstein_polynomial.py` (Appendix A).

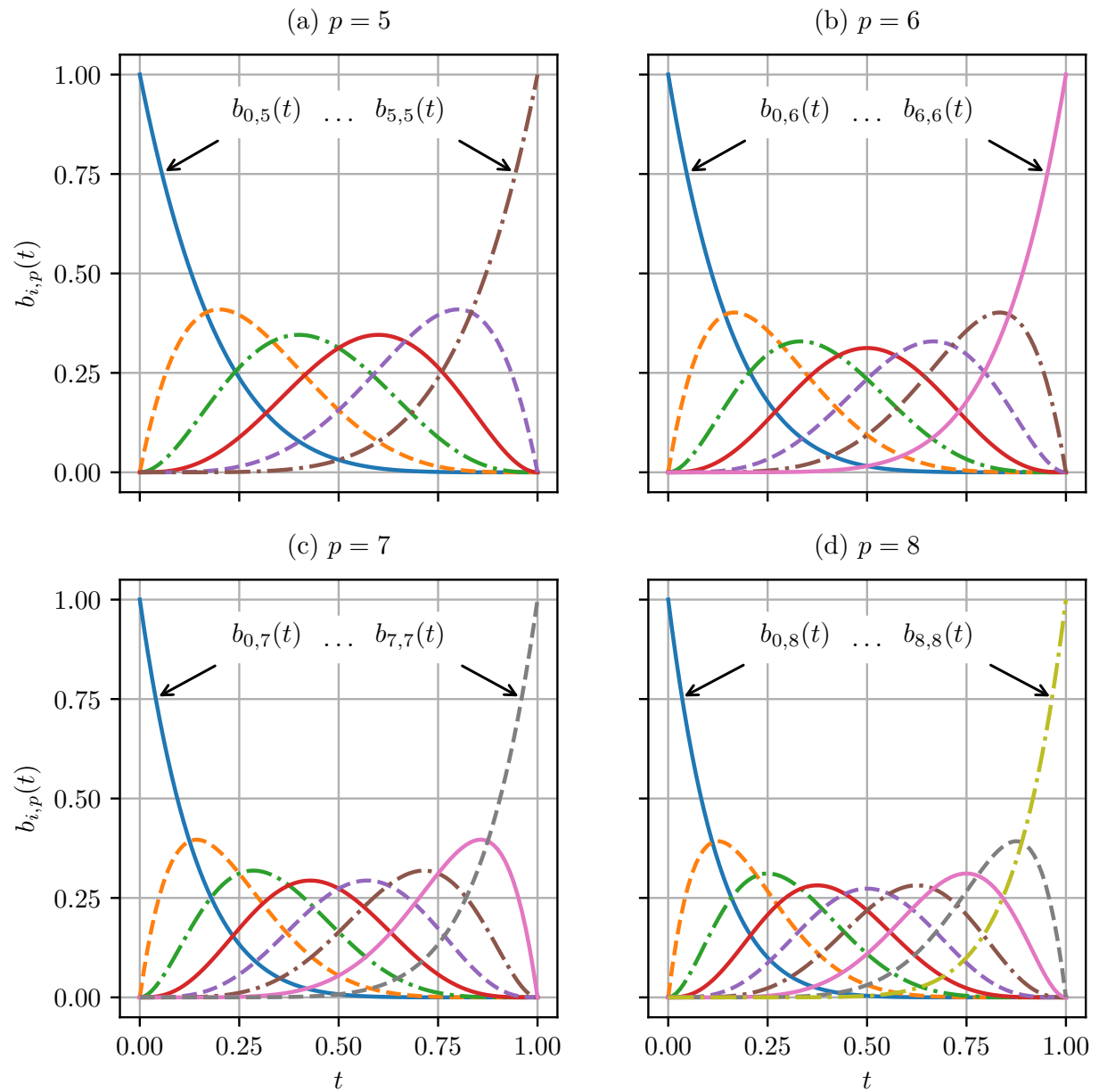


Figure 2.4: Bernstein polynomials for Bézier (a) $p = 5$, (b) $p = 6$, (c) $p = 7$, and (d) $p = 8$ curves. Reference: `fig/bernstein_extended.py`, `bernstein_extended.py` (Appendix A).

2.3 Bézier Surfaces

The Bézier curve was parameterized by t . For the Bézier surface, we have two parameters, u and v . More to come.

Chapter 3

Local Methods

Local methods mean that any change in a control point does not necessarily affects the *entire* parameterization. This is generally very desirable, as changes can be focused on a specific region, without disrupting the balance of the parameterization. However, local methods are more complicated than non-local methods, and require definition over intervals, as well be seen below.

3.1 General Splines

Let a collection of $n+1$ points (x_i, y_i) for $i = 0, 1, \dots, n$ approximate the function $y = f(x)$ over the domain $[x_0, x_n]$. In total, there are $n+1$ points and n intervals. Let $S(x)$ be the piecewise continuous spine that threads through each of the points (x_i, y_i) . For $x \in [x_i, x_{i+1}]$ and $i = 0, 1, \dots, n-1$, the spline takes on n interval definitions as follows:

$$S_i(x) \triangleq a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 + \dots + z_i(x - x_i)^p. \quad (3.1)$$

where constant coefficients¹ multiply successive powers² of $(x - x_i)$, summed to an arbitrary polynomial power p . This is referred to as the **interval spline definition** to polynomial degree p . Below we consider linear splines ($p = 1$), quadratic splines ($p = 2$), and cubic splines ($p = 3$).

3.2 Linear Splines

For linear splines, the interval spline definition is

$$S_i(x) \triangleq a_i + b_i(x - x_i) \quad \text{for } x \in [x_i, x_{i+1}] \text{ and } i = 0, 1, \dots, n-1. \quad (3.2)$$

For any collection of points (x_i, y_i) , all coefficients a_i and b_i for $i = 0, 1, \dots, n-1$ can be found using the **zeroth derivative continuity constraint**, which creates a spline that connects to each

¹Here the z_i is arbitrary, meant to denote the last coefficient, but not necessarily the twenty-sixth coefficient.

²The powers are non-negative integers.

point at the beginning and end of each interval, thus

$$S_i(x_i) = y_i \quad \text{and} \quad S_i(x_{i+1}) = y_{i+1} \quad \text{for } i = 0, 1, \dots, n-1. \quad (3.3)$$

Here we see there are two constraints per interval i and two unknowns, a_i and b_i , per interval i . The unknowns can be found directly by evaluating the interval spline definition with each of the two constraint equations. In turn, these are, for $i = 0, 1, \dots, n-1$,

$$S_i(x_i) = a_i + b_i(x_i - x_i) \stackrel{\text{set}}{=} y_i \implies a_i = y_i, \text{ and} \quad (3.4)$$

$$S_i(x_{i+1}) = a_i + b_i(x_{i+1} - x_i) \stackrel{\text{set}}{=} y_{i+1} \implies b_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}. \quad (3.5)$$

No additional continuity constraints can be created since the linear spline has only C^0 continuity, meaning no derivatives of the function $S_i(x)$ are continuous.

Example 4.

Let $y = f(x) = \sin(\pi x)$ for $x \in [0, 1]$ be approximated by a linear spline and a collection of three points equally spaced on the domain. Note $n + 1 = 3$, thus $n = 2$, indicating three points and two intervals. Table 3.1 summarizes the spline data. Figure 3.1(a) shows the $\sin$ function and the linear spline interpolations. $\square$

Table 3.1: Linear spline approximation of $y = f(x) = \sin(\pi x)$ using three points and two intervals.

i	x_i	y_i	$\implies$	a_i	b_i	$\implies$	$S_i(x)$
0	0/2	0		0	2		$2x$
1	1/2	1		1	-2		$1 - 2(x - 1/2)$
2	2/2	0		—	—		—

Example 5.

Repeat the previous example, but now with a collection of five points equally spaced on the domain. Thus $n + 1 = 5$, thus $n = 4$, indicating five points and four intervals. Table 3.2 summarizes the spline data. Figure 3.1(b) shows the sin function and the linear spline interpolations. $\square$

Table 3.2: Linear spline approximation of $y = f(x) = \sin(\pi x)$ using five points and four intervals.

i	x_i	y_i	$\Rightarrow$	a_i	b_i	$\Rightarrow$	$S_i(x)$
0	0/4	0		0	$2\sqrt{2}$		$2\sqrt{2}x$
1	1/4	$\sqrt{2}/2$		$\sqrt{2}/2$	$4 - 2\sqrt{2}$		$\sqrt{2}/2 + (4 - 2\sqrt{2})(x - 1/4)$
2	2/4	1		1	$2\sqrt{2} - 4$		$1 + (2\sqrt{2} - 4)(x - 2/4)$
3	3/4	$\sqrt{2}/2$		$\sqrt{2}/2$	$-2\sqrt{2}$		$\sqrt{2}/2 - 2\sqrt{2}(x - 3/4)$
4	4/4	0		—	—		—

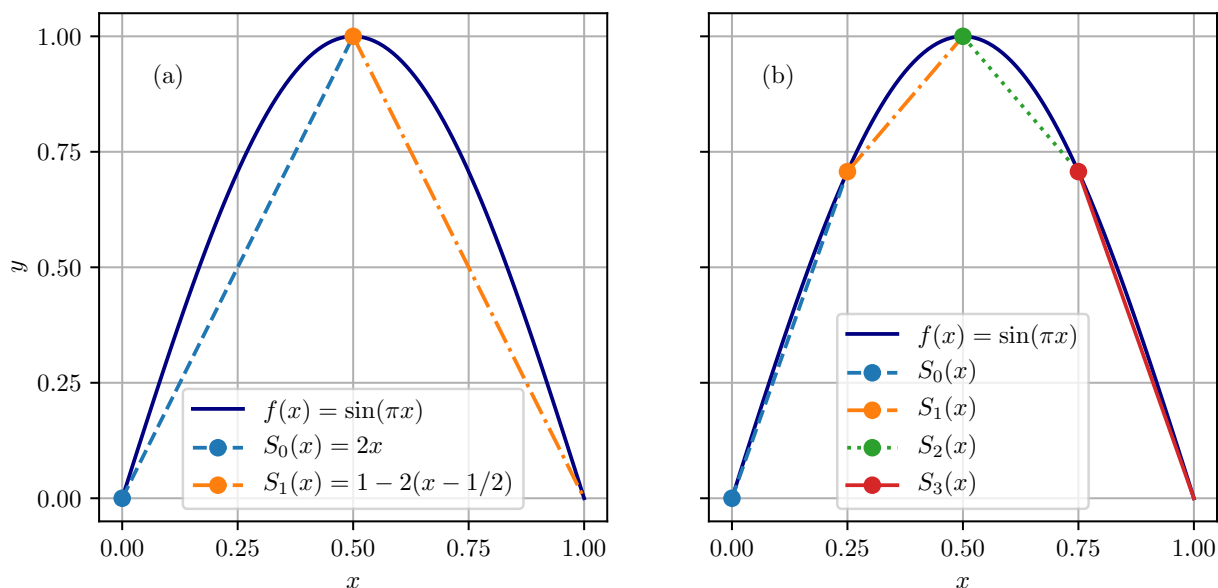


Figure 3.1: Linear spline example (a) with three points and (b) with five points, see Table 3.2 for individual definitions of $S_i(x)$. Reference: fig/linear_spline.py.

3.3 Quadratic Splines

For quadratic splines, the interval spline definition is

$$S_i(x) \triangleq a_i + b_i(x - x_i) + c_i(x - x_i)^2 \quad \text{for } x \in [x_i, x_{i+1}] \text{ and } i = 0, 1, \dots, n-1. \quad (3.6)$$

3.4 Cubic Splines

Cubic splines are useful because of their inherent smoothness. The cubic spline, its first derivative, and its second derivative are all continuous. Starting with the cubic spline's third derivative, discontinuity arises.

Let a collection of $n + 1$ points (x_i, y_i) for $i = 0, 1, \dots, n$ approximate the function $y = f(x)$ over the domain $[x_0, x_n]$. In total, there are $n + 1$ points and n intervals. Let $S(x)$ be the piecewise continuous spline, cubic in this case, that threads through each of the points (x_i, y_i) . The spline takes on n interval definitions as follows:

$$S_i(x) \triangleq a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad \text{for } x \in [x_i, x_{i+1}] \text{ and } i = 0, 1, \dots, n-1. \quad (3.7)$$

The n foregoing definitions, with the coefficients a_i, b_i, c_i, d_i , give rise to $4n$ (currently unknown) parameters used to define the complete spline $S(x)$. Note that maintain its cubic nature, the spline must have $d_i \neq 0$.

Next we endow the spline with constraints on continuity and differentiability, which allow the $4n$ unknown parameters to be solved. First, function continuity requires that the spline, on a particular interval, thread through collection of points (x_i, y_i) and the beginning and end of each interval. Thus,

$$S_i(x_i) = y_i \text{ and } S_i(x_{i+1}) = y_{i+1} \quad \text{for } i = 0, 1, \dots, n-1. \quad (3.8)$$

This is the **continuity constraint**, which gives rise to $2n$ conditions³ and creates a spline that is piecewise continuous. Next, consider two **differentiability constraints**, which state that the first and second derivatives at a point x_i are continuous. That is, the first and second derivatives approaching a point x_i from the interval to the left side of the point must match the respective first and second derivatives approaching the point x_i from the interval to the right side of the point:

$$S'_{i-1}(x_i) = S'_i(x_i) \text{ and } S''_{i-1}(x_i) = S''_i(x_i) \quad \text{for } i = 1, 2, \dots, n-1. \quad (3.9)$$

Because they apply only to the interior points (and not the endpoints x_0 and x_n), the differentiability constraints give rise to $2(n-1)$ constraints.⁴

³Two conditions per interval times n intervals yields $2n$ conditions.

⁴Two conditions per interior point times $n-1$ interior points yields $2(n-1)$ conditions.

Finally, two more conditions are necessary to completely define the spline $S(x)$. The two conditions are obtained by specifying differentiability conditions on the two spline endpoints x_0 and x_n . Frequently, the first derivative is set to the known value of the function's derivatives at the endpoints,

$$S'_0(x_0) = f'(x_0) \text{ and } S'_{n-1}(x_n) = f'(x_n). \quad (3.10)$$

This is defined as a **clamped spline**. Alternatively, a **natural spline** defines the second derivatives to be zero at the endpoints,

$$S''_0(x_0) = 0 \text{ and } S''_{n-1}(x_n) = 0. \quad (3.11)$$

Finally, a **not-a-knot** spline requires no additional constraints on the endpoint derivatives,⁵ but requires that the *third* derivative be continuous at x_1 and x_{n-1} .

3.5 B-Splines

A **B-spline** is a spline composed of two or more Bézier curves, joined end-to-end, subject to certain restrictions, which will be discussed below. The “B” in B-spline stands for **basis**, as the spline curves, defined over an interval, utilize control points that are interpolated with basis functions. In contrast, general splines (*e.g.*, the linear, quadratic, and cubic splines discussed in the previous sections) use data points that are interpolated directly, without basis functions.

B-splines are **parametric**, so the functional form for a general B-spline curve $\mathbf{S} \in \mathbb{R}^3$, will be written explicitly as $\mathbf{S}(t; \mathbf{x})$, similar to the notation introduced with the Bézier curves in Section 2.2. Similar to the Bézier compact notation discussed previously, B-spline curves are typically written in terms of the parametric variable t , without the explicit notation of the space coordinates $\mathbf{x}$ of the control points, *i.e.*, $\mathbf{P}_i(\mathbf{x}) = \langle x_i, y_i, z_i \rangle$.

Similar to the definition of a general Bézier curve $\mathbf{C}_p(t)$ of degree p in (2.32), the general form for a B-spline $\mathbf{S}_p(t)$ of degree p , with $n + 1$ (not necessarily $p + 1$, as with Bézier curves) control points $\mathbf{P}_i$, $i = 0, 1, \dots, n$, is given by

$$\mathbf{S}_p(t) \triangleq \sum_{i=0}^n N_{i,p}(t) \mathbf{P}_i, \quad (3.12)$$

where $N_{i,p}$ is the **normalized basis function** and $\mathbf{P}_i \in \mathbb{R}^3$ are the control points $\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_n$. The normalized basis functions are defined by the **Cox-de Boor recursion formulas**:

$$\text{for } p = 0 : \quad N_{i,0}(t) \triangleq \begin{cases} 1 & \text{if } t_i \leq t < t_{i+1}, \\ 0 & \text{otherwise.} \end{cases} \quad (3.13)$$

⁵This has no effect on the original continuity constraint on the endpoints, that $S_0(x_0) = y_0$ and $S_{n-1}(x_n) = y_n$.

$$\text{for } p \geq 1 : \quad N_{i,p}(t) \triangleq \frac{t - t_i}{t_{i+p} - t_i} N_{i,p-1}(t) + \frac{t_{i+p+1} - t}{t_{i+p+1} - t_{i+1}} N_{i+1,p-1}(t). \quad (3.14)$$

The structure of the Cox-de Boor recursion formulas is shown below through illustrations of linear ($p = 1$), quadratic ($p = 2$), and cubic ($p = 3$) examples.

Example 6.

Linear B-spline ($p = 1$):

$$\text{for } p = 1 : \quad N_{i,1}(t) = \frac{t - t_i}{t_{i+1} - t_i} N_{i,0}(t) + \frac{t_{i+2} - t}{t_{i+2} - t_{i+1}} N_{i+1,0}(t) \quad (3.15)$$

The linear forms are shown in the middle subfigure of Figure 3.2. $\square$

Example 7.

Quadratic B-spline ($p = 2$):

$$\text{for } p = 2 : \quad N_{i,2}(t) = \frac{t - t_i}{t_{i+2} - t_i} N_{i,1}(t) + \frac{t_{i+3} - t}{t_{i+3} - t_{i+1}} N_{i+1,1}(t) \quad (3.16)$$

The linear forms are shown in the bottom subfigure of Figure 3.2. $\square$

Example 8.

Cubic B-spline ($p = 3$):

$$\text{for } p = 3 : \quad N_{i,3}(t) = \frac{t - t_i}{t_{i+3} - t_i} N_{i,2}(t) + \frac{t_{i+4} - t}{t_{i+4} - t_{i+1}} N_{i+1,2}(t) \quad (3.17)$$

$\square$

The discrete values (*e.g.*, $t_i, t_{i+1}, t_{i+2}, t_{i+3}, t_{i+4}, t_{i+p}$, and t_{i+p+1}) are referred to as **knots**, which mark the termination points, beginning or end, of an interval for the parameter t . The collection of all knots is called a **knot vector**, designated by $\mathbf{T}$. We define the knot vector $\mathbf{T}$ as a non-decreasing set of non-negative integer coordinates in parameter space,

$$\mathbf{T} \triangleq \langle t_0, t_1, t_2, \dots, t_m \rangle. \quad (3.18)$$

The values for parameter t are $t \in \mathbb{R} \subset [t_0, t_m]$. In total, there are $(m + 1)$ knots in knot vector $\mathbf{T}$.

Example 9.

A uniform knot vector containing 10 knots is written as

$$\mathbf{T} = \langle 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \rangle, \text{ with } t \in \mathbb{R} \subset [0, 9]. \quad (3.19)$$

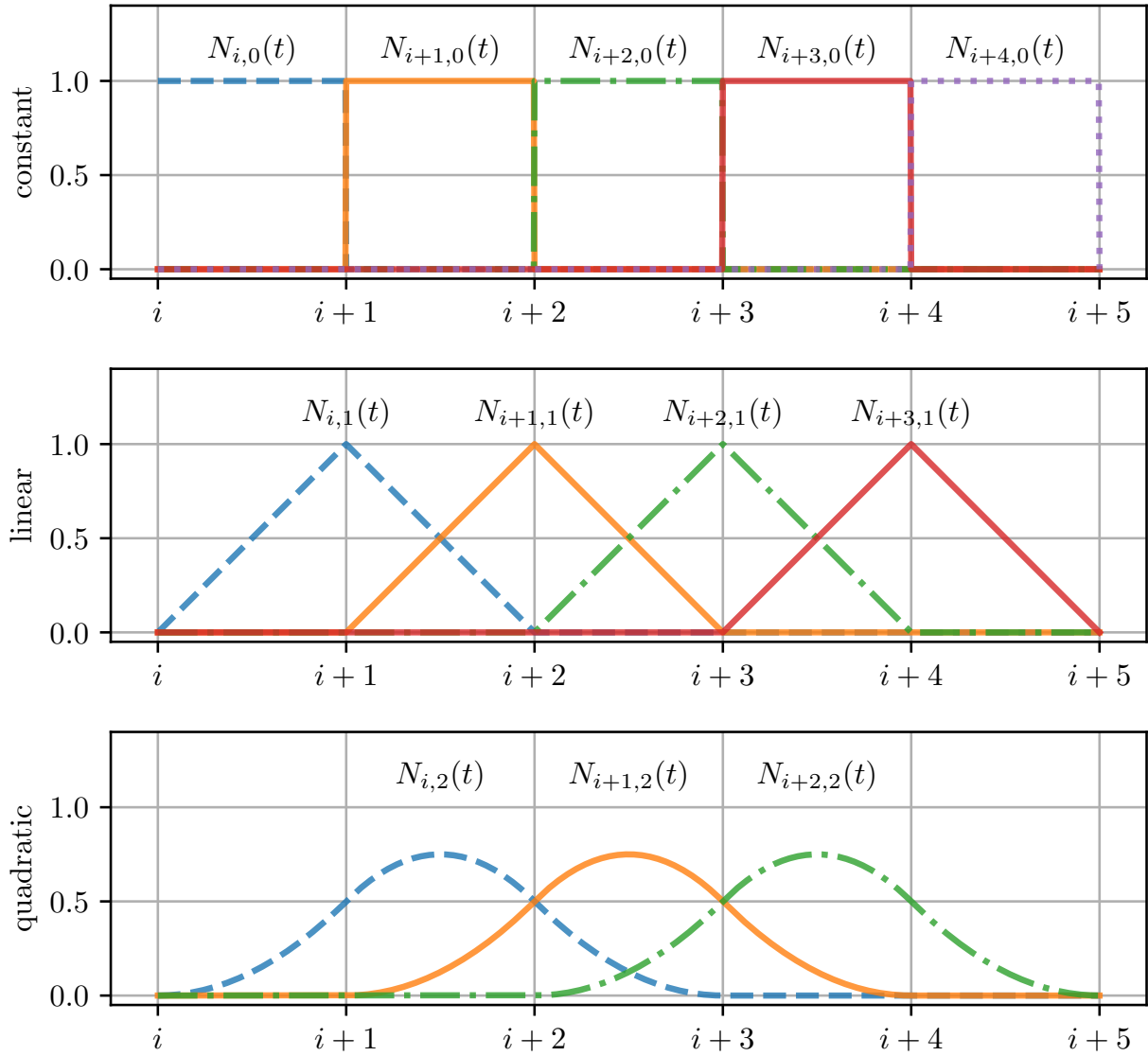


Figure 3.2: Normalized basis functions of the Cox-de Boor recursion formula for constant, linear, and quadratic forms. Reference: `fig/bspline.py`.

□

To construct the B-spline $\mathbf{S}_p(t)$ with normalized basis functions $N_{i,p}(t)$, $(n+1)$ control points $\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_n$, and degree p , exactly $(m+1)$ knots must be specified. The number of knots, $(m+1)$, is related to the number of control points, $(n+1)$, and the degree, p , as

$$m = (n + 1) + p. \quad (3.20)$$

Presently, we consider a knot vector in one dimension only. Additional dimensions will be discussed later. Also, for now we will limit the discussion to a **uniform knot vector**, wherein the distance between any two consecutive knots is the same,

$$\Delta t = t_1 - t_0 = t_2 - t_1 = \cdots = t_m - t_{m-1} = 1 = \text{constant}. \quad (3.21)$$

Later, we will consider two additional concepts: *repeated* knots and *non-uniform* knots.⁶

⁶The variable knot-to-knot distance will create a **non-uniform knot vector**, which is the “non-uniform” of NURBs (non-uniform rational B-splines).

Appendices

Appendix A

Code Listing

```
1  #!/usr/bin/env python3
2  # bernstein_polynomial.py
3  import math
4  import sys
5  import numpy as np
6
7  def bernstein_polynomial(i, p, nti):
8      """
9      Computes the Bernstein polynomial coefficient for
10     control point i with
11     polynomial degree p
12     for a 1D parameter array t in interval [0, 1] broken into
13     nti number of equidistance time intervals.
14     """
15     if i >= 0 and p >= 1 and nti >= 2 and i <= p:
16         t = np.linspace(0, 1, nti+1)
17         bp = math.factorial(p) / \
18             (math.factorial(i) * math.factorial(p-i)) * t**i * (1-t)**(p-i)
19         return bp
20     else:
21         print(f'Input (i, p, nti) = ({i}, {p}, {nti}) is out of range.')
22         print('i is non-negative integer 0, 1, 2, ... p')
23         print('p is integer >= 1')
24         print('nti is integer >= 2')
25         return None
```

```
1  #!/usr/bin/env python3
2  # bernstein_polynomial_test.py
3  """
4  This module is a unit test of the bernstein_polynomial implementation.
5
6  To run
7  £ python bernstein_polynomial_test.py           # terse interaction
8  £ python -m unittest bernstein_polynomial_test  # default interaction
9  £ python -m unittest -v bernstein_polynomial_test # verbose interaction
10 """
```

```

11  # standard library imports
12  import sys
13  #
14  import numpy as np
15  from unittest import TestCase, main
16  #
17  import bernstein_polynomial as bp
18
19  class TestBernstein(TestCase):
20
21      @classmethod
22      def setUpClass(cls):
23          cls._TOL = 1e-6 # tolerance
24          cls._nti = 4 # number of time intervals
25          # t, e.g., four intervals, five evaluation points
26          cls._t = np.linspace(0, 1, cls._nti+1)
27          cls._verbosity = 0
28
29      @classmethod
30      def same(cls, a, b):
31          same_to_tolerance = False
32          l2norm_diff = np.linalg.norm(a - b)
33
34          if cls._verbosity > 0:
35              print(f'array a = {a}')
36              print(f'array b = {b}')
37              print(f'l2norm_diff = {l2norm_diff}')
38
39          if np.abs(l2norm_diff) < cls._TOL:
40              same_to_tolerance = True
41
42          return same_to_tolerance
43
44      def test_000_b01(self):
45          known = 1 - self._t
46          i, p = 0, 1
47          calc = bp.bernstein_polynomial(i, p, self._nti)
48          self.assertTrue(self.same(known, calc))
49
50      def test_001_b11(self):
51          known = self._t
52          i, p = 1, 1
53          calc = bp.bernstein_polynomial(i, p, self._nti)
54          self.assertTrue(self.same(known, calc))
55
56      def test_003_b02(self):
57          known = (1 - self._t)**2
58          i, p = 0, 2
59          calc = bp.bernstein_polynomial(i, p, self._nti)
60          self.assertTrue(self.same(known, calc))
61
62      def test_004_b12(self):
63          known = 2 * self._t * (1 - self._t)
64          i, p = 1, 2
65          calc = bp.bernstein_polynomial(i, p, self._nti)

```

```

66         self.assertTrue(self.same(known, calc))
67
68     def test_005_b22(self):
69         known = (self._t)**2
70         i, p = 2, 2
71         calc = bp.bernstein_polynomial(i, p, self._nti)
72         self.assertTrue(self.same(known, calc))
73
74     def test_006_b03(self):
75         known = (1 - self._t)**3
76         i, p = 0, 3
77         calc = bp.bernstein_polynomial(i, p, self._nti)
78         self.assertTrue(self.same(known, calc))
79
80     def test_007_b13(self):
81         known = 3 * self._t * (1 - self._t)**2
82         i, p = 1, 3
83         calc = bp.bernstein_polynomial(i, p, self._nti)
84         self.assertTrue(self.same(known, calc))
85
86     def test_008_b23(self):
87         known = 3 * (self._t)**2 * (1 - self._t)
88         i, p = 2, 3
89         calc = bp.bernstein_polynomial(i, p, self._nti)
90         self.assertTrue(self.same(known, calc))
91
92     def test_009_b33(self):
93         known = (self._t)**3
94         i, p = 3, 3
95         calc = bp.bernstein_polynomial(i, p, self._nti)
96         self.assertTrue(self.same(known, calc))
97
98     def test_010_b04(self):
99         known = (1 - self._t)**4
100        i, p = 0, 4
101        calc = bp.bernstein_polynomial(i, p, self._nti)
102        self.assertTrue(self.same(known, calc))
103
104    def test_011_b14(self):
105        known = 4 * self._t * (1 - self._t)**3
106        i, p = 1, 4
107        calc = bp.bernstein_polynomial(i, p, self._nti)
108        self.assertTrue(self.same(known, calc))
109
110    def test_012_b24(self):
111        known = 6 * (self._t)**2 * (1 - self._t)**2
112        i, p = 2, 4
113        calc = bp.bernstein_polynomial(i, p, self._nti)
114        self.assertTrue(self.same(known, calc))
115
116    def test_013_b34(self):
117        known = 4 * (self._t)**3 * (1 - self._t)
118        i, p = 3, 4
119        calc = bp.bernstein_polynomial(i, p, self._nti)
120        self.assertTrue(self.same(known, calc))

```

```

121
122     def test_014_b44(self):
123         known = (self._t)**4
124         i, p = 4, 4
125         calc = bp.bernstein_polynomial(i, p, self._nti)
126         self.assertTrue(self.same(known, calc))
127
128 if __name__ == '__main__':
129     main() # calls unittest.main()

```

```

1  #!/usr/bin/env python3
2  # bernstein_extended.py
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from matplotlib import rc
6  from matplotlib.ticker import AutoMinorLocator, MultipleLocator
7
8  import bernstein_polynomial as bp
9
10 LATEX = 1
11 SERIALIZE = 0
12
13 if LATEX:
14     rc('font', **{'family': 'serif', 'serif': ['Computer Modern Roman']})
15     rc('text', usetex=True)
16
17 nts = 101 # number of time steps, include t0 as a step
18 nti = nts - 1 # number of time intervals
19 t = np.linspace(0, 1, nts) # parameterization
20
21 # fig = plt.figure(figsize=(6, 6)) # x in inches wide, y inches tall
22 fig = plt.figure(figsize=(6.5, 6.5)) # x in incmes wide, y inches tall
23 ax2 = fig.add_subplot(2, 2, 3, aspect=1)
24 ax0 = fig.add_subplot(2, 2, 1, aspect=1)
25 ax3 = fig.add_subplot(2, 2, 4, aspect=1, sharey=ax2)
26 ax1 = fig.add_subplot(2, 2, 2, aspect=1, sharey=ax0, sharex=ax3)
27
28 # globals
29 ap = dict(arrowstyle='->') # arrow properties
30 lax, lay = 0.5, 1.1
31 b0_x = 0.32
32 b0_y = 0.87
33 dx = 0.008
34 annotate_dict = dict(arrowprops=ap,
35     ha='center',
36     va='bottom',
37     backgroundcolor='white')
38 text_dict = dict(ha='center',
39     va='bottom',
40     backgroundcolor='white')
41 linestyle_tuple = ('solid', 'dashed', 'dashdot')
42
43 p = 5
44 for q in np.arange(p+1):

```

```

45     ls = linestyle_tuple[q % len(linestyle_tuple)]
46     ax0.plot(t, bp.bernstein_polynomial(q, p, nti), linestyle=ls)
47
48     ax0.annotate(r'$b_{0,5}(t)$', xy=(0.06-dx, 0.75), xytext=(b0_x, b0_y), **annotate_dict)
49     ax0.text(lax, b0_y, r'$\ldots$', **text_dict)
50     ax0.annotate(r'$b_{5,5}(t)$', xy=(0.94+dx, 0.75), xytext=(1-b0_x, b0_y), **annotate_dict)
51
52     ax0.xaxis.set_major_locator(MultipleLocator(0.25))
53     ax0.yaxis.set_major_locator(MultipleLocator(0.25))
54     ax0.set_ylabel(r'$b_{i,p}(t)$')
55     ax0.text(lax, lay, r'(a) $p=5$', **text_dict)
56     ax0.grid()
57     plt.setp(ax0.get_xticklabels(), visible=False)
58
59     p = 6
60     for q in np.arange(p+1):
61         ls = linestyle_tuple[q % len(linestyle_tuple)]
62         ax1.plot(t, bp.bernstein_polynomial(q, p, nti), linestyle=ls)
63
64         ax1.annotate(r'$b_{0,6}(t)$', xy=(0.06-2*dx, 0.75), xytext=(b0_x, b0_y), **annotate_dict)
65         ax1.text(lax, b0_y, r'$\ldots$', **text_dict)
66         ax1.annotate(r'$b_{6,6}(t)$', xy=(0.94+2*dx, 0.75), xytext=(1-b0_x, b0_y), **annotate_dict)
67         ax1.xaxis.set_major_locator(MultipleLocator(0.25))
68         ax1.yaxis.set_major_locator(MultipleLocator(0.25))
69         ax1.text(lax, lay, r'(b) $p=6$', **text_dict)
70         ax1.grid()
71         plt.setp(ax1.get_xticklabels(), visible=False)
72         plt.setp(ax1.get_yticklabels(), visible=False)
73
74     p = 7
75     for q in np.arange(p+1):
76         ls = linestyle_tuple[q % len(linestyle_tuple)]
77         ax2.plot(t, bp.bernstein_polynomial(q, p, nti), linestyle=ls)
78
79         ax2.annotate(r'$b_{0,7}(t)$', xy=(0.06-3*dx, 0.75), xytext=(b0_x, b0_y), **annotate_dict)
80         ax2.text(lax, b0_y, r'$\ldots$', **text_dict)
81         ax2.annotate(r'$b_{7,7}(t)$', xy=(0.94+3*dx, 0.75), xytext=(1-b0_x, b0_y), **annotate_dict)
82
83         ax2.xaxis.set_major_locator(MultipleLocator(0.25))
84         ax2.yaxis.set_major_locator(MultipleLocator(0.25))
85         ax2.set_xlabel(r'$t$')
86         ax2.set_ylabel(r'$b_{i,p}(t)$')
87         ax2.text(lax, lay, r'(c) $p=7$', **text_dict)
88         ax2.grid()
89
90     p = 8
91     for q in np.arange(p+1):
92         ls = linestyle_tuple[q % len(linestyle_tuple)]
93         ax3.plot(t, bp.bernstein_polynomial(q, p, nti), linestyle=ls)
94
95         ax3.annotate(r'$b_{0,8}(t)$', xy=(0.06-4*dx, 0.75), xytext=(b0_x, b0_y), **annotate_dict)
96         ax3.text(lax, b0_y, r'$\ldots$', **text_dict)
97         ax3.annotate(r'$b_{8,8}(t)$', xy=(0.94+4*dx, 0.75), xytext=(1-b0_x, b0_y), **annotate_dict)
98
99         ax3.xaxis.set_major_locator(MultipleLocator(0.25))

```

```
100 ax3.yaxis.set_major_locator(MultipleLocator(0.25))
101 ax3.set_xlabel(r'$t$')
102 ax3.text(lax, lay, r'(d) $p=8$', **text_dict)
103 ax3.grid()
104 plt.setp(ax3.get_yticklabels(), visible=False)
105
106 plt.show()
107
108 if SERIALIZE:
109     fig.savefig("bernstein_extended.pdf", bbox_inches="tight")
```

Bibliography

- [Bartels 1995] Bartels RH, Beatty JC, Barsky BA. An introduction to splines for use in computer graphics and geometric modeling. Morgan Kaufmann; 1995 Sep 15.
- [Cottrell 2009] Cottrell JA, Hughes TJ, Bazilevs Y. Isogeometric analysis: toward integration of CAD and FEA. John Wiley & Sons; 2009 Aug 11.
- [Heinrich 2016] Heinrich G. Image segmentation using DIGITS 5, NVIDIA Developer Blog. <https://devblogs.nvidia.com/image-segmentation-using-digits-5>; 2016 Nov 10.
- [IRCAD 2019] IRCAD France. 3D-IRCADB 01 database, 3D CT-scans of 10 women and 10 men. <https://www.ircad.fr/research/3d-ircadb-01/>; accessed 2019 Aug 29.
- [Piegl 2012] Piegl L, Tiller W. The NURBS book: edition 2. Springer Science & Business Media; 2012 Dec 6.
- [Rogers 2000] Rogers DF. An introduction to NURBS: with historical perspective. Elsevier; 2000 Aug 22.
- [Ronneberger 2015] Ronneberger O, Fischer P, Brox T. U-net: Convolutional networks for biomedical image segmentation. In International Conference on Medical image computing and computer-assisted intervention. <https://arxiv.org/abs/1505.04597>; 2015 Oct 5 (pp. 234-241). Springer, Cham.
- [Shiach 2015] Shiach J. Bézier Curves. Mathematics of Computer Graphics and Virtual Environments. <https://youtu.be/2HvH9cmHbG4>; 2015 Mar 09.
- [Shiach 2015b] Shiach J. B-Splines. Mathematics of Computer Graphics and Virtual Environments. <https://youtu.be/qhQrRCJ-mVg>; 2015 Mar 13.